Modelo documental y continuidad

Integrantes: Exequiel Arancibia y Joaquin Romero. Informe de cinco páginas; fuentes y evidencias adjuntas.

Objetivo. Representar pedidos existentes de Food Store en MongoDB y comprobar sus resultados contra PostgreSQL. Se resuelven las siete consignas de la guía, conservando identificadores, estados, cantidades y precios históricos.

1. Subconjunto y correspondencia de claves

Se utiliza **foodstore**, la base acumulada de las unidades 4 a 6: FNBC, vistas e índices de U4, seguridad de U5 y continuidad verificada en U6. El cliente se llama **usuario** en nuestro esquema. Se seleccionan los pedidos **1, 2, 3, 4 y 5**, ya existentes; no se crean registros en PostgreSQL.

Origen PostgreSQL	Destino MongoDB / significado
pedido.id (PK)	_id Long: mismo identificador del pedido
pedido.usuario_id -> usuario.id	cliente.usuario_id Long + alias de presentación
detalle_pedido PK (pedido_id, producto_id)	lineas[] dentro de cada pedido; producto_id Long
detalle_pedido.producto_id -> producto.id	Nombre del producto al exportar; no es un nombre histórico
fecha, estado, forma_pago, total	Date UTC, estados originales y total Decimal128

```
foodstore_u7.pedidos = {
  _id: Long, fecha: Date, estado: String, forma_pago: String,
  cliente: { usuario_id: Long, alias: String },
  lineas: [{ producto_id: Long, nombre_al_migrar: String,
    cantidad: Int32, precio_unitario: Decimal128, subtotal: Decimal128 }],
  total: Decimal128
}
```

2-3. Por qué se embeben los datos

Las líneas pertenecen al pedido y normalmente se consultan juntas. Embebidas, el pedido completo se recupera de una colección, sin reconstruirlo con uniones. Cada documento conserva su total y los precios de **detalle_pedido.precio_unitario**; no toma el precio actual del catálogo.

El cliente esencial se representa con su ID y una etiqueta nueva, por ejemplo **Cliente 1**. No se exportan nombre, apellido, email, teléfono ni hash. El alias no sustituye ni modifica al usuario original. Cliente y nombre del producto son una copia al momento de exportación; no se implementa sincronización automática. [1]

Migración e integridad verificadas

Consignas 2 y 3 - Ejecución real del 10/11/2026, con salida y archivos exactos adjuntos.

Procedimiento ejecutado

1. PostgreSQL 17.11: transacción **REPEATABLE READ READ ONLY**; selección explícita de los cinco IDs y exportación Extended JSON. Se comprueban las siete líneas y la igualdad entre cada total y la suma de subtotales.

2. Se conservan IDs como Long, cantidades Int32 e importes Decimal128, sin pasar dinero a Double. Las cinco fechas no contienen fracciones submilisegundo: Date UTC conserva sus instantes; por ejemplo, 01/03/2026 12:00 UTC-03 corresponde a 15:00Z.

3. MongoDB Community **8.0.34** y mongosh **2.14.1**: creación de **foodstore_u7.pedidos** con validador estricto e **insertMany()**. Se usaron binarios oficiales portátiles con SHA256 oficial coincidente, autenticación y acceso exclusivo por loopback.

4. Se compara el contenido BSON de los cinco documentos contra el EJSON exportado. Se crean índices por **estado** y **lineas.producto_id**. Este último es multikey por indexar un arreglo; no se midió una mejora temporal. [2]

```
Salida: motor=8.0.34; base=foodstore_u7; colección=pedidos
documentos=5; lineas=7; contenido_igual_exportacion=true
tamaños BSON por ID: [440,305,311,440,306] bytes
integridad_origen: lineas_invalidas=0; pedidos_total_incorrecto=0
psql exit=0; mongosh exit=0
```

Dos reglas de integridad conservadas

Regla	Aplicación en la migración
1. Dominio e identidad	Cantidad entera > 0, precio histórico > 0, IDs originales y producto sin repetir en un pedido. _id único y enums reales del origen.
2. Coherencia monetaria	subtotal = cantidad x precio_unitario; total = suma(subtotales). $jsonSchema + $expr, validationAction:error. No recalcular precios desde producto.

Pruebas negativas reales. Total incoherente, cantidad cero (ajustando subtotal/total para aislar esa regla) y producto repetido con total correcto: rechazados con **código 121**. ID duplicado: **11000**. Persisten exactamente cinco documentos y ningún registro inválido. El validador expresa estas reglas; no equivale a todas las FK, transiciones y permisos del sistema relacional. [3]

Conservación del origen. Las huellas de filas completas antes/después coinciden: categoria 2; producto 50.000; usuario 20.001; pedido 225.000; detalle_pedido 499.997. PostgreSQL no recibe escrituras de este práctico. Las evidencias incluyen esos controles y hashes de las fuentes ejecutadas.

Consultas y equivalencia de resultados

Consignas 4 y 5 - Todas las comparaciones se restringen a los mismos pedidos [1,2,3,4,5].

4. Cuatro consultas ejecutadas en MongoDB

```
a) pedidos.find({}).sort({_id:1})
b) pedidos.find({estado:'ENTREGADO'},{_id:1,estado:1})
c) pedidos.find({'lineas.producto_id':Long('1')},{_id:1})
d) pedidos.find({total:{$gt:Decimal128('3000.00')}},{_id:1,total:1})
```

a) ID	Estado	Total	Productos / cantidades
1	ENTREGADO	2800.00	1 x 2; 2 x 1
2	ENTREGADO	1500.00	3 x 1
3	ENTREGADO	3150.00	1 x 3
4	ENTREGADO	6200.00	2 x 4; 3 x 2
5	CONFIRMADO	1050.00	1 x 1

La consulta **a** devuelve documentos completos, incluidos ID y estado; su salida integra la evidencia adjunta. **b** devuelve [1,2,3,4]. **c** busca el producto 1 dentro del arreglo y devuelve [1,3,5]. **d** devuelve [3,4], con totales 3150.00 y 6200.00. Se elige **3000.00** porque separa dos pedidos de mayor importe de los tres restantes; es un umbral exploratorio de la muestra, no una regla comercial.

5. Equivalentes ejecutados en PostgreSQL

```
SELECT id FROM public.pedido
WHERE id IN (1,2,3,4,5) AND estado='ENTREGADO';

SELECT p.id FROM public.pedido p
WHERE p.id IN (1,2,3,4,5) AND EXISTS (
  SELECT 1 FROM public.detalle_pedido d
  WHERE d.pedido_id=p.id AND d.producto_id=1);
```

Filtro	PostgreSQL	MongoDB	IDs en ambos / coinciden
ENTREGADO	4	4	[1,2,3,4] / Sí
Producto 1	3	3	[1,3,5] / Sí

No se detectaron diferencias. **EXISTS** conserva una fila por pedido, como find() devuelve un documento por pedido. Comparar MongoDB con los 225.000 pedidos del origen completo daría una diferencia de alcance; por eso ambas consultas SQL usan la muestra explícita. No fue necesario modificar datos para obtener equivalencia.

Evaluación de tecnología y asistencia de IA

Consignas 6 y 7 - Propuestas de IA contrastadas con documentación oficial y ejecución propia.

6. Consulta realizada y trazabilidad

Se consultó **Codex, agente IA de comparación**, el 10/11/2026. El prompt describió pedido, detalle_pedido, usuario y producto; indicó claves foráneas, subtotal generado y triggers, el acceso frecuente al pedido completo y la necesidad de conservar coherencia de cantidades, precios históricos y total. Se solicitaron dos afirmaciones verificables y una recomendación, evitando suponer benchmarks o que MongoDB carece de transacciones.

El prompt exacto y la respuesta original breve se conservan en **docs/ia/consulta_respuesta.md**, adjunto al PDF. Se compartió un resumen del esquema y del problema; no filas ni datos personales. La respuesta de IA se utilizó como propuesta y se comprobó antes de adoptar decisiones.

7. Contraste de dos afirmaciones concretas

Afirmación 1 de IA: un pedido con líneas embebidas puede recuperarse como un documento sin unir colecciones. **Contraste:** la documentación oficial de embebidos describe ese acceso conjunto [1]. En el laboratorio, la consulta a recuperó los cinco pedidos con sus siete líneas desde pedidos; el ID 1 contiene dos productos. El almacenamiento y el filtro por producto usan el arreglo del mismo documento. **Resultado: aceptada** como diferencia de estructura. No se afirma que MongoDB sea más rápido: cinco documentos y equivalencia de IDs no constituyen un benchmark.

Afirmación 2 de IA: una escritura documental es atómica, pero embeber líneas no garantiza automáticamente coherencia matemática. **Contraste:** MongoDB documenta la atomicidad por documento y admite transacciones entre documentos en despliegues compatibles [4]. Su validación de esquema permite tipos/rangos y rechaza documentos inválidos [3]. Se implementaron reglas explícitas: el total incoherente y la cantidad cero fueron rechazados con código 121. **Resultado: aceptada.** Atomicidad y validez de reglas de negocio son propiedades distintas. Nuestro servidor standalone no ejecuta transacciones multidocumento.

Comparación aplicada a Food Store

Criterio	PostgreSQL acumulado	MongoDB de este laboratorio
Pedido completo	JOIN o proyección JSON; integridad y triggers ya implementados	Un documento con líneas; copia orientada a lectura
Coherencia	FK, restricciones, subtotal generado y triggers existentes [5]	Validador explícito; no replica toda la política relacional
Costo operativo	Se mantiene una fuente de verdad	Combinar exige sincronización y controlar copias obsoletas

Conclusión, reproducción y fuentes

Cierre del informe - Código y evidencias adjuntos al PDF y disponibles como archivos separados.

Conclusión (menos de 200 palabras)

Conviene mantener PostgreSQL como fuente principal de Food Store. El primer criterio es la integridad: las claves foráneas, restricciones, subtotal generado y triggers ya protegen relaciones y reglas que no se trasladan automáticamente al documento. El segundo es el costo de operación: sumar otro motor obliga a definir sincronización, manejo de cambios y recuperación de copias obsoletas. MongoDB representa bien el pedido completo con sus líneas y permite filtrar por estado o producto sin reconstruir el agregado. La muestra migró sin diferencias de identificadores ni cantidades, conservando importes decimales y precios históricos. Esto valida el modelo y las consultas del laboratorio, pero no demuestra una mejora de rendimiento ni justifica sustituir PostgreSQL. Una proyección MongoDB podría incorporarse para lecturas específicas si mediciones y requisitos reales la justifican, manteniendo PostgreSQL como autoridad y definiendo un mecanismo verificable de actualización. En este práctico se implementa una copia puntual de cinco pedidos, sin sincronización ni cambio del sistema principal.

Cómo reproducir y comprobar

1. Ejecutar consultas_postgresql.sql con psql sobre foodstore acumulada; requiere la conexión propia del laboratorio. **2.** Extraer consultas_mongodb.js y pedidos_migrados.ejson a una misma carpeta. Desde ella ejecutar mongosh con la conexión autenticada propia y **--file consultas_mongodb.js**. **3.** Comprobar cinco documentos, siete líneas, IDs b/c iguales y cuatro rechazos esperados. Los archivos no contienen credenciales.

Repositorio de trabajo: Git local Arancibia_Romero_U7, sin remoto. Commit de fuentes: **8178b01**. Asistencia de IA declarada y validada mediante fuentes oficiales y pruebas reales. Los commits técnicos se identifican como Codex. README y evidencia incluyen comandos, alcance, salidas y hashes.

Fuentes oficiales y material de la asignatura

[1] MongoDB: Embedded Data in Your MongoDB Schema

[2] MongoDB: Multikey Indexes

[3] MongoDB: Schema Validation

[4] MongoDB: Atomicity and Transactions

[5] PostgreSQL 17: Constraints

Material UTN FRM: Trabajo Práctico Unidad 7 y apunte NoSQL y nuevas formas de almacenamiento, edición octubre 2026, apartados 4-10, 17 y 20-22. Se adaptó su ejemplo a usuario, lineas y estados del esquema real. Antecedentes: entregas U4-U6 del equipo. Consulta oficial: 10/11/2026.

Adjuntos exactos: JS, SQL, EJSON, README, respuesta IA y evidencias de ejecución/conservación. Las tres fuentes necesarias para reproducir también se entregan como archivos separados.